

AI DOCUMENT ANALYZER

Project Submission Document

Project Type	Data Science / Artificial Intelligence Project
Application	Flask-Based Document Processing and Analysis Web Application
Developer	Gokulakkannan
Technologies	Python, Flask, SQLite, SQLAlchemy, Tesseract OCR
Supported Files	PDF, DOCX, TXT, JPG, JPEG, PNG

Project submission content covering the system, implementation, testing, limitations and future enhancements.

1. Abstract

AI Document Analyzer is a Flask-based web application designed to automate basic document processing and analysis. Authenticated users can upload PDF, DOCX, TXT, JPG, JPEG and PNG files. The system extracts text, performs OCR for images, extracts keywords and structured information, and generates a rule-based summary. Users can search documents, re-analyze files, delete documents and export analysis results as TXT or PDF reports. SQLite and SQLAlchemy are used for data storage and Flask-Login manages authentication.

2. Problem Statement

Manual document processing can be time-consuming when users need to extract text, identify important keywords, find contact information and prepare a quick summary. Image-based documents also require OCR before their content can be analyzed. This project combines these operations into one web-based workflow.

3. Objectives

- Provide a web interface for uploading and managing documents.
- Support PDF, DOCX, TXT, JPG, JPEG and PNG formats.
- Extract text from digital documents and images.
- Perform OCR on supported image documents.
- Extract frequently occurring keywords.
- Identify email addresses and phone numbers when detected.
- Generate a concise rule-based summary.
- Search uploaded documents by filename or extracted content.
- Re-analyze previously uploaded documents.
- Delete documents and their stored files.
- Export analysis results as TXT and PDF.
- Provide authenticated user accounts and protected document access.
- Store user and document metadata using SQLite and SQLAlchemy.

4. Main Features

Feature	Description
Authentication	Registration, validation, password hashing, login, logout and sessions.
Document Upload	Upload supported document and image formats.
Text Extraction	Extract text from PDF, DOCX and TXT files.
OCR	Extract text from JPG, JPEG and PNG using Tesseract OCR.
Keyword Extraction	Identify frequently occurring relevant words.
Information Extraction	Detect email addresses and phone numbers.
Summarization	Generate a concise rule-based summary.
Search / Filter	Search by filename or extracted content.
Re-analysis	Run extraction and analysis again.
Delete	Remove the document record and stored file.
Export	Download analysis results as TXT or PDF.
Profile	Display username, email and account creation information.

5. Technology Stack

- Python and Flask
- SQLite and Flask-SQLAlchemy
- Flask-Login for authentication
- PyMuPDF for PDF processing
- python-docx for DOCX processing
- Tesseract OCR with pytesseract
- Pillow for image processing
- ReportLab for PDF report generation
- pytest for automated testing
- HTML and CSS with Flask templates for the frontend

6. System Architecture

The browser communicates with Flask routes. Routes handle authentication, document management and profile operations. Service modules perform text extraction, OCR, keyword extraction, information extraction and summarization. SQLAlchemy communicates with the SQLite database, while uploaded files are stored in the application file storage.

Layer	Components
User Interface	Browser, HTML, CSS, Flask templates
Application	Flask routes, authentication, request handling
Services	Text extraction, OCR, keywords, information, summary
Data	SQLAlchemy and SQLite
File Storage	Uploaded document files

7. Database Design

The application uses SQLite with SQLAlchemy. A one-to-many relationship connects users and documents: one user can have multiple documents, and each document belongs to one authenticated user.

Table	Fields
users	id, username, email, password_hash, created_at
documents	id, filename, filepath, extracted_text, uploaded_at, user_id

8. Module Description

Authentication Module

Registration, email validation, password confirmation, duplicate checking, password hashing, login and logout.

Document Management

Upload, list, search, analyze, re-analyze, delete and export.

Text Extraction

Extract text from PDF, DOCX and TXT.

OCR Module

Process supported image files using Tesseract OCR.

Keyword Extraction

Use text processing and frequency counting to identify top keywords.

Information Extraction

Detect email addresses and phone numbers.

Summarization

Generate a concise rule-based summary.

Profile Module

Display authenticated user information.

9. Implementation Workflow

- User registers or logs in.
- User selects a supported document.
- The file is uploaded and stored.
- The system identifies the file type.
- Text is extracted directly for PDF, DOCX and TXT files.
- OCR is applied to supported image files.
- Extracted text is stored with the document record.
- Keywords, structured information and summary are generated.
- The analysis result is displayed.
- The user can search, re-analyze, delete or export the result.

10. Testing and Results

The application was tested through browser-based manual testing and automated pytest tests. The automated test suite completed with 3 passed tests out of 3 collected tests.

Test Area	Result
Registration and validation	Pass
Login / Logout	Pass
PDF / DOCX / TXT extraction	Pass
Image OCR	Pass for tested clear image
Keyword extraction	Pass
Information extraction	Pass
Summary generation	Pass
Search / Clear Search	Pass
Re-analysis	Pass
Delete	Pass
TXT / PDF Export	Pass
Profile	Pass
Automated pytest	3 / 3 Passed

11. Limitations

- OCR accuracy depends on image quality, resolution, fonts and layout.
- Complex or highly designed documents can produce OCR recognition errors.

- The current summarization module is rule-based and is not an LLM-based summarizer.
- The local setup uses SQLite and local file storage.
- A separate scanned-PDF OCR pipeline is not currently implemented.
- The current application is primarily intended as a local development/demo system.

12. Future Enhancements

- Add OCR support for scanned PDF documents.
- Improve OCR preprocessing and document-layout handling.
- Add stronger semantic summarization using NLP/LLM methods where appropriate.
- Extract dates, organizations, skills and addresses.
- Add document categories and advanced filtering.
- Add pagination for large document collections.
- Add cloud storage and production database support.
- Add stronger production security and deployment configuration.
- Expand automated test coverage.

13. GitHub Repository

Repository: github.com/chmlgokul/AI-Document-Analyzer

The repository contains the application source code, requirements, README, tests and project documentation.

14. Demo / Presentation Flow

- 1 Introduce the project and its purpose.
- 2 Show the home page and main features.
- 3 Register or log in.
- 4 Upload a clear sample document or resume image.
- 5 Show extracted text, keywords, information and summary.
- 6 Open My Documents and demonstrate search.
- 7 Show re-analysis if required.
- 8 Demonstrate TXT/PDF export.
- 9 Briefly explain the database and technology stack.
- 10 Mention testing results and OCR limitations.
- 11 Conclude with future enhancements.

15. Conclusion

AI Document Analyzer provides an integrated workflow for document upload, text extraction, OCR, keyword extraction, information extraction, summarization, document management and report export. The implemented functionality was tested through browser-based manual testing and automated pytest testing. The project provides a functional foundation that can be extended with stronger NLP, improved OCR, cloud storage and production deployment.

Prepared for Project Submission